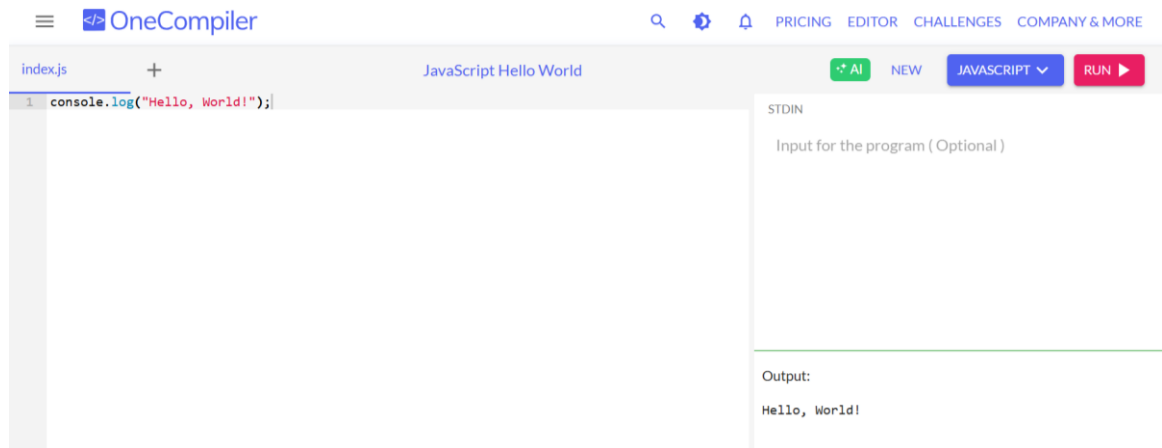


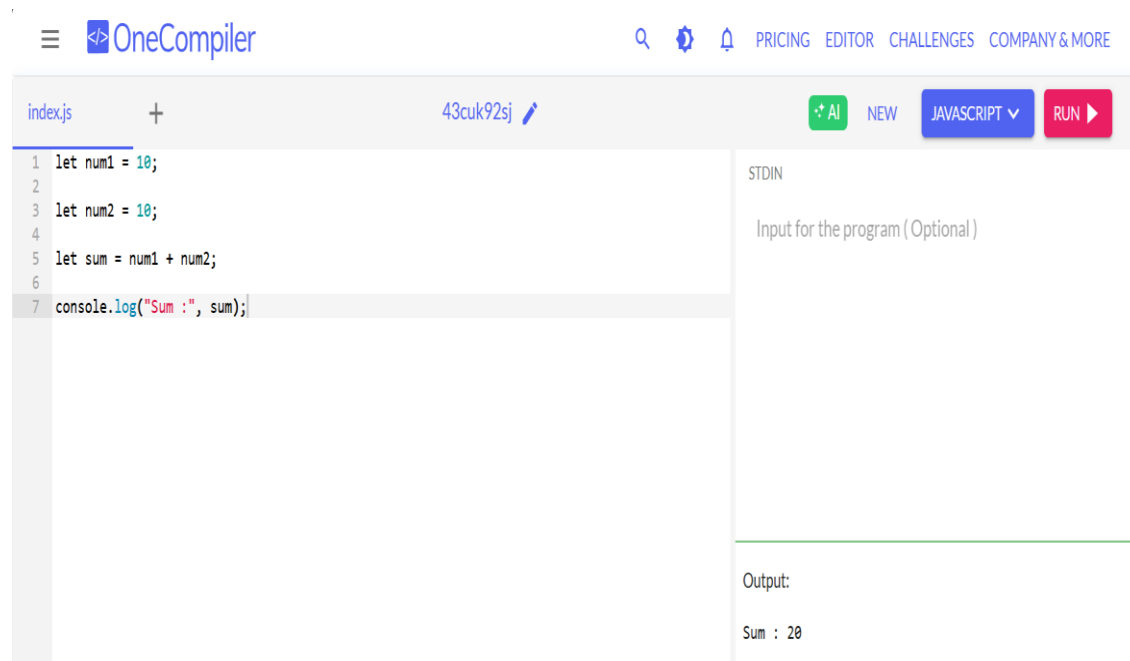
JavaScript Assignment

1. Create a simple web page displaying "Hello, World!" using JavaScript.



2. Write a function to sum two numbers.

JavaScript Program to Add Two Numbers



Using function

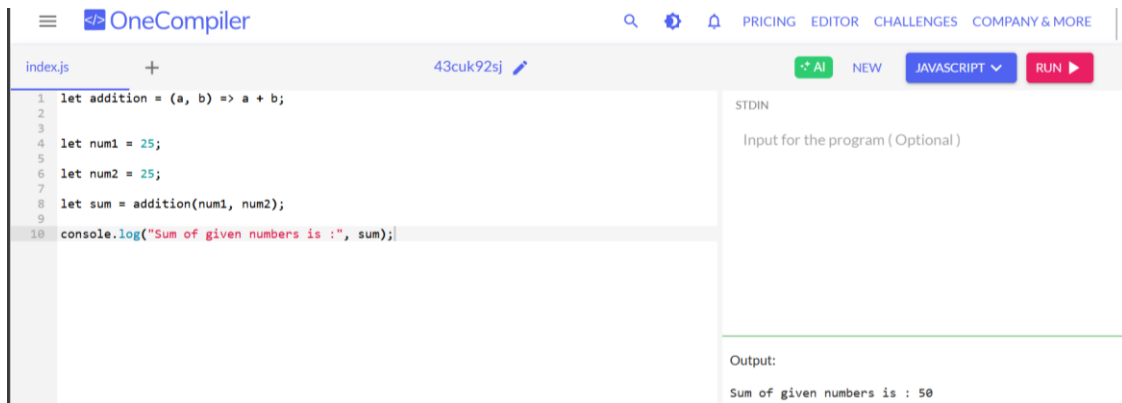


The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code defines a function 'additionFunction(a, b)' that returns the sum of 'a' and 'b'. It then initializes 'num1' to 5 and 'num2' to 10, calls the function, and logs the result. The output shows 'Sum of given numbers is : 15'.

```
1 function additionFunction(a, b) {  
2   return a + b;  
3 }  
4  
5  
6  
7  
8 let num1 = 5;  
9  
10 let num2 = 10;  
11  
12 let sum = additionFunction(num1, num2);  
13  
14 console.log("Sum of given numbers is :", sum);
```

Output:
Sum of given numbers is : 15

Using Arrow function



The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code uses an arrow function 'addition = (a, b) => a + b;'. It initializes 'num1' to 25 and 'num2' to 25, calls the function, and logs the result. The output shows 'Sum of given numbers is : 50'.

```
1 let addition = (a, b) => a + b;  
2  
3  
4 let num1 = 25;  
5  
6 let num2 = 25;  
7  
8 let sum = addition(num1, num2);  
9  
10 console.log("Sum of given numbers is :", sum);
```

Output:
Sum of given numbers is : 50

Using Addition Assignment (+=) Operator



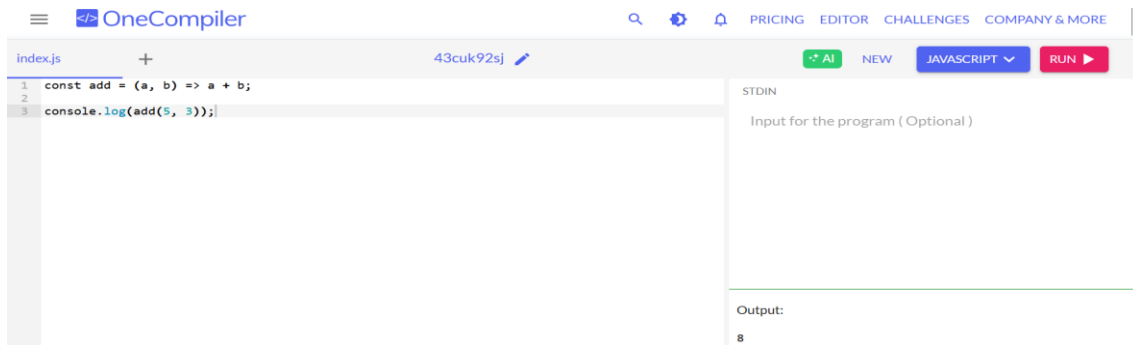
The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code initializes 'num1' to 15 and 'num2' to 10. It then uses the addition assignment operator 'num1 += num2;' to add 'num2' to 'num1'. A comment indicates this is equivalent to 'num1 = num1 + num2'. Finally, it logs the value of 'num1'. The output shows 'Sum of the given number is : 25'.

```
1 let num1 = 15;  
2  
3 let num2 = 10;  
4  
5  
6 // Equivalent to num1 = num1 + num2  
7  
8 num1 += num2;  
9  
10 console.log("Sum of the given number is :", num1);
```

Output:
Sum of the given number is : 25

3. Convert a regular function to an arrow function.

Arrow functions in JavaScript

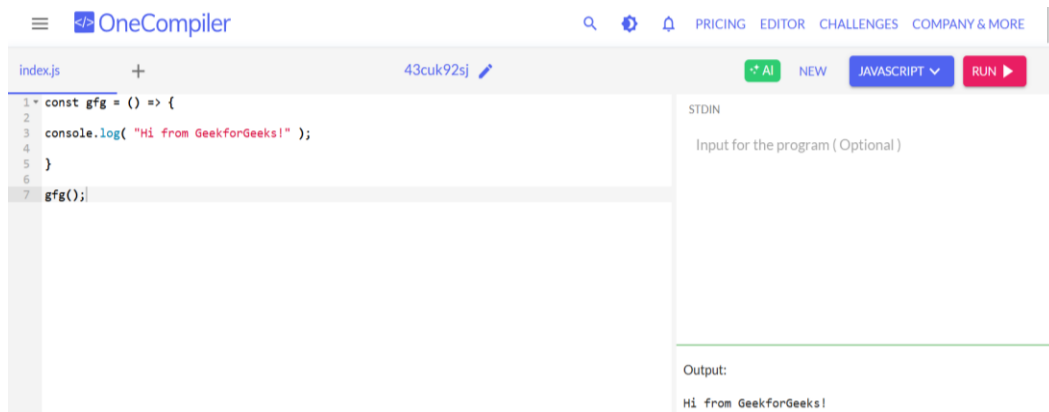


The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code defines an arrow function 'add' that takes two parameters, 'a' and 'b', and returns their sum. It then calls 'console.log' with 'add(5, 3)'. The output on the right is '8'.

```
1 const add = (a, b) => a + b;  
2  
3 console.log(add(5, 3));
```

Output:
8

4. Arrow Function without Parameters

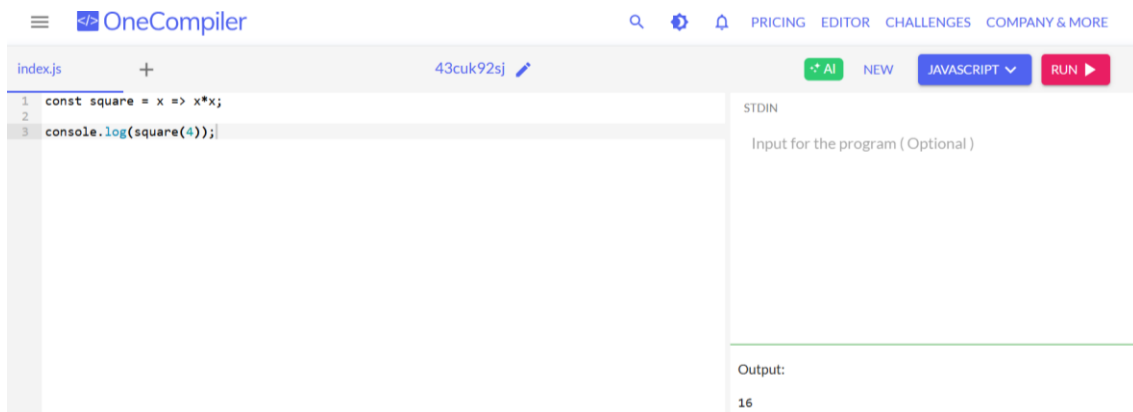


The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code defines an arrow function 'gfg' without parameters, which logs a string to the console. It then calls 'gfg()'. The output on the right is 'Hi from GeekforGeeks!'.

```
1 const gfg = () => {  
2   console.log( "Hi from GeekforGeeks!" );  
3 }  
4  
5  
6  
7 gfg();
```

Output:
Hi from GeekforGeeks!

5. Arrow Function with Single Parameters

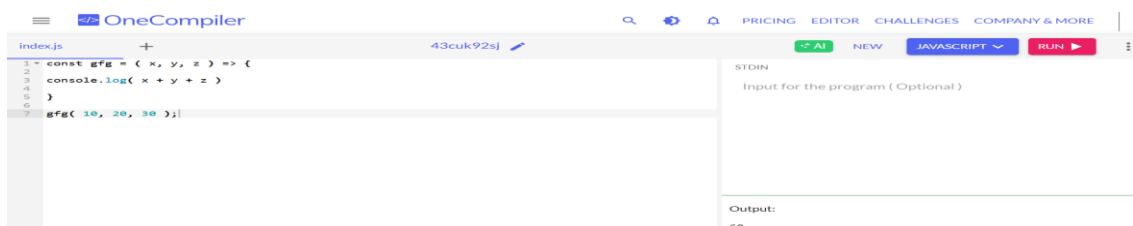


The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code defines an arrow function 'square' that takes a single parameter 'x' and returns 'x*x'. It then calls 'console.log' with 'square(4)'. The output on the right is '16'.

```
1 const square = x => x*x;  
2  
3 console.log(square(4));
```

Output:
16

6. Arrow Function with Multiple Parameters.

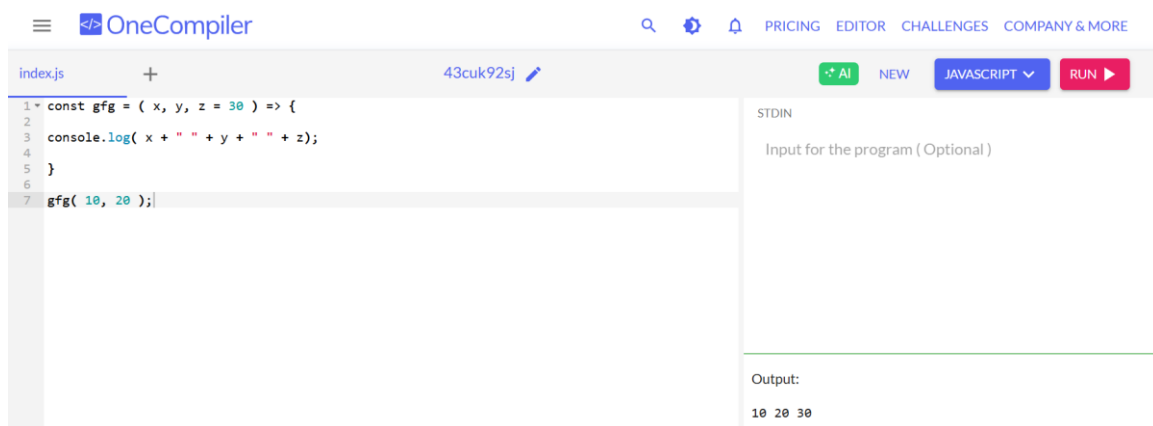


The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code defines an arrow function 'gfg' that takes three parameters 'x', 'y', and 'z', and returns their sum. It then calls 'gfg' with the values 10, 20, and 30. The output on the right is '60'.

```
1 const gfg = (x, y, z) => {  
2   console.log( x + y + z )  
3 }  
4  
5  
6  
7 gfg( 10, 20, 30 );
```

Output:
60

4. Arrow Function with Default Parameters



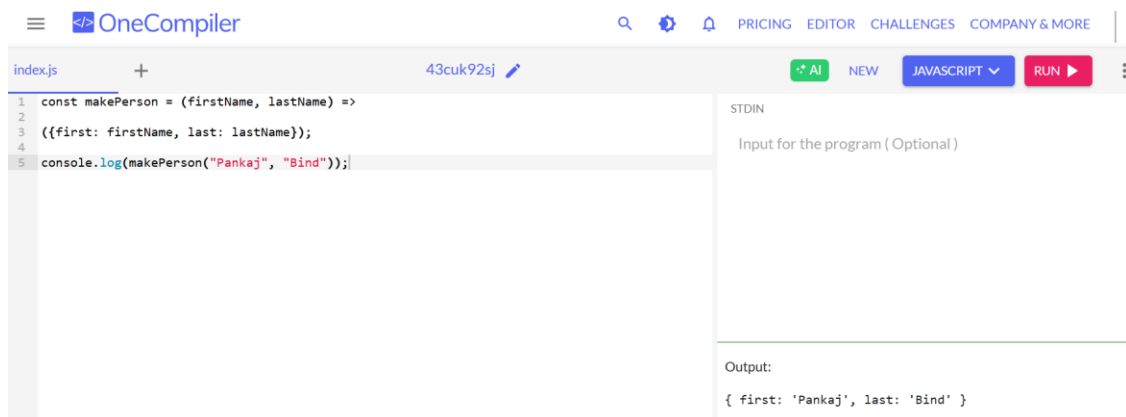
The screenshot shows the OneCompiler interface. The code editor contains the following JavaScript code:

```
1 const gfg = ( x, y, z = 30 ) => {  
2   console.log( x + " " + y + " " + z);  
3 }  
4  
5  
6  
7 gfg( 10, 20 );
```

The output section displays the result of the function call:

```
Output:  
10 20 30
```

5. Return Object Literals



The screenshot shows the OneCompiler interface. The code editor contains the following JavaScript code:

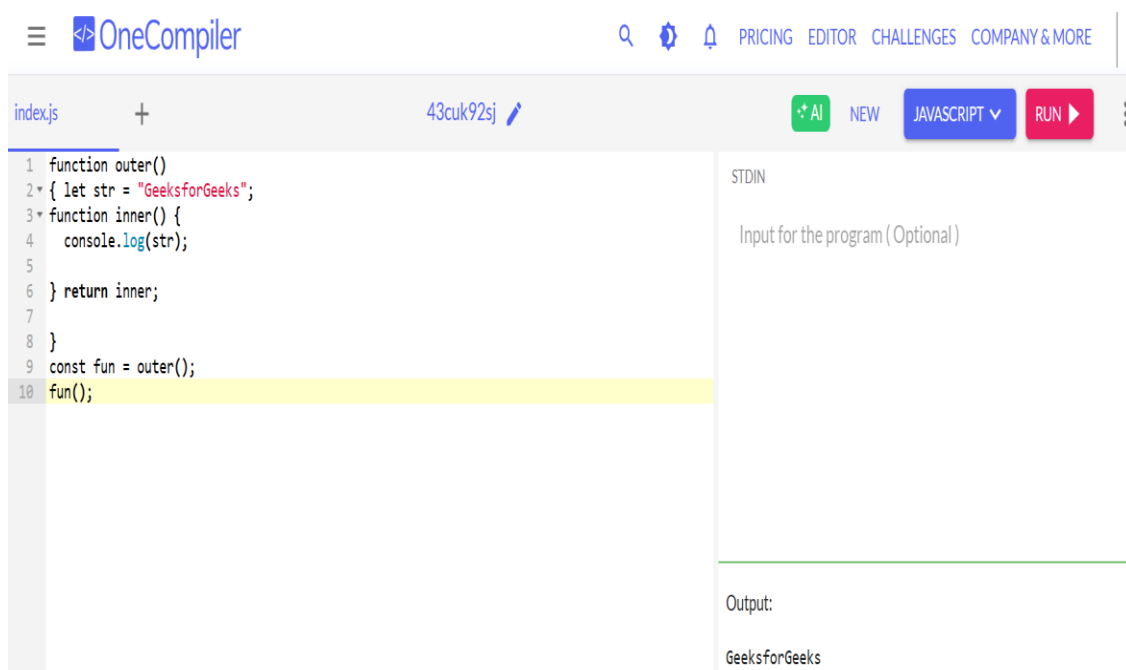
```
1 const makePerson = (firstName, lastName) =>  
2   ({first: firstName, last: lastName});  
3  
4 console.log(makePerson("Pankaj", "Bind"));
```

The output section displays the result of the function call:

```
Output:  
{ first: 'Pankaj', last: 'Bind' }
```

4. Create a counter function using closures.

How to use a closure to create a private counter in JavaScript?



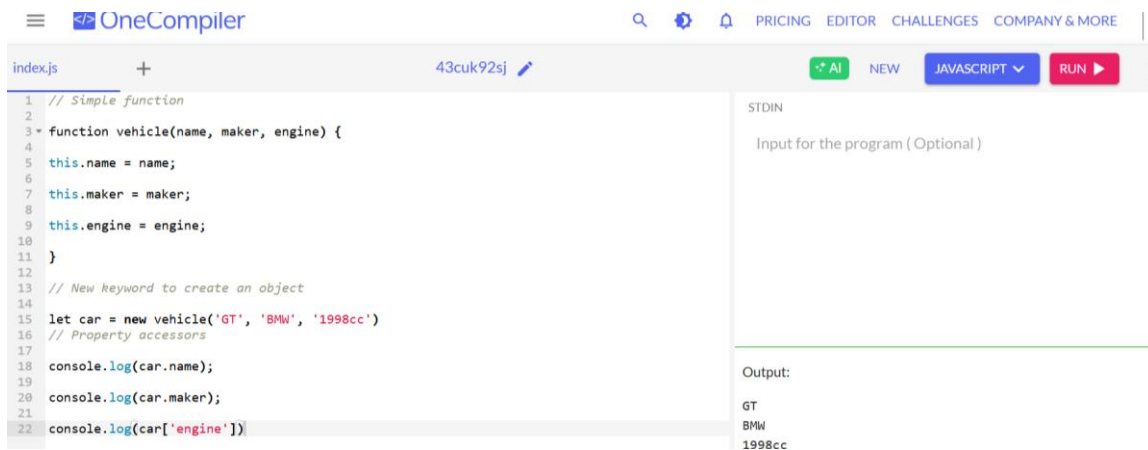
The screenshot shows the OneCompiler interface. The code editor contains the following JavaScript code:

```
1 function outer()  
2 { let str = "GeeksforGeeks";  
3   function inner() {  
4     console.log(str);  
5   }  
6   return inner;  
7 }  
8  
9 const fun = outer();  
10 fun();
```

The output section displays the result of the function call:

```
Output:  
GeeksforGeeks
```

5. Define an object representing a car with properties and a method.



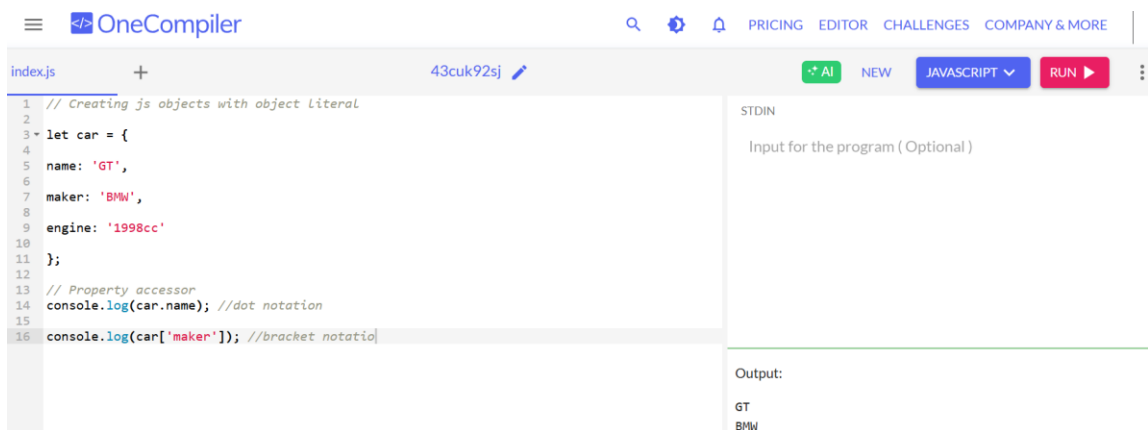
The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code defines a function 'vehicle' that takes 'name', 'maker', and 'engine' as arguments. It sets 'this.name', 'this.maker', and 'this.engine' to the respective arguments. Then, it creates a new object 'car' using 'new vehicle('GT', 'BMW', '1998cc')'. Finally, it logs 'car.name', 'car.maker', and 'car[engine]' to the console. The output on the right shows 'GT', 'BMW', and '1998cc' on separate lines.

```
1 // Simple function
2
3 function vehicle(name, maker, engine) {
4
5   this.name = name;
6
7   this.maker = maker;
8
9   this.engine = engine;
10
11 }
12
13 // New keyword to create an object
14
15 let car = new vehicle('GT', 'BMW', '1998cc')
16 // Property accessors
17
18 console.log(car.name);
19
20 console.log(car.maker);
21
22 console.log(car['engine'])
```

Output:

```
GT
BMW
1998cc
```

Using object literals

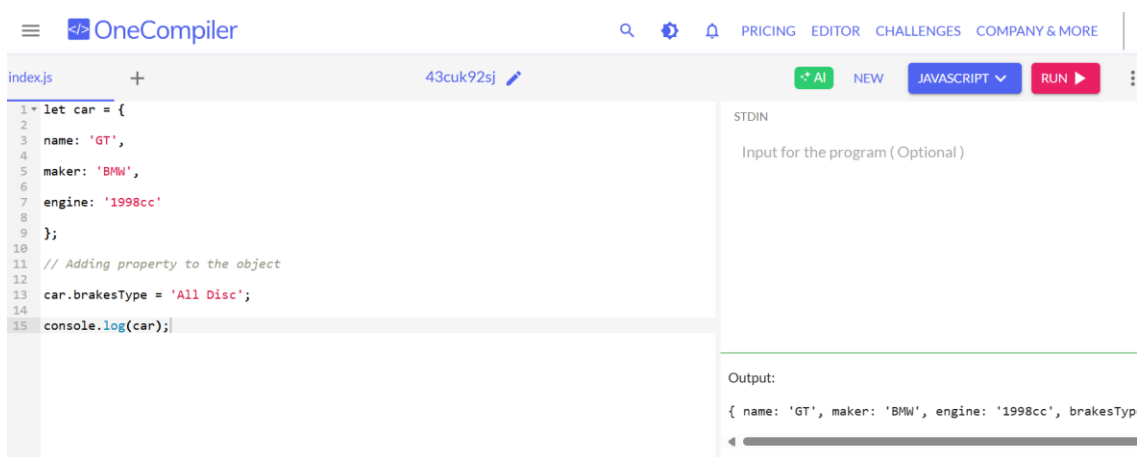


The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code creates an object 'car' using an object literal with properties 'name', 'maker', and 'engine'. It then logs 'car.name' using dot notation and 'car['maker']' using bracket notation. The output on the right shows 'GT' and 'BMW' on separate lines.

```
1 // Creating js objects with object Literal
2
3 let car = {
4
5   name: 'GT',
6
7   maker: 'BMW',
8
9   engine: '1998cc'
10
11 };
12
13 // Property accessor
14 console.log(car.name); //dot notation
15
16 console.log(car['maker']); //bracket notation
```

Output:

```
GT
BMW
```



The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code creates an object 'car' using an object literal with properties 'name', 'maker', and 'engine'. It then adds a new property 'brakesType' to the object. Finally, it logs the entire object 'car'. The output on the right shows the full object structure: '{ name: 'GT', maker: 'BMW', engine: '1998cc', brakesTyp'.

```
1 let car = {
2
3   name: 'GT',
4
5   maker: 'BMW',
6
7   engine: '1998cc'
8
9 };
10
11 // Adding property to the object
12
13 car.brakesType = 'All Disc';
14
15 console.log(car);
```

Output:

```
{ name: 'GT', maker: 'BMW', engine: '1998cc', brakesTyp
```

The screenshot shows the OneCompiler web interface. The editor contains the following JavaScript code:

```
1 // Adding methods to the car object
2
3 let car = {
4
5   name : 'GT',
6
7   maker : 'BMW',
8
9   engine : '1998cc',
10
11  start : function(){
12    console.log('Starting the engine...');
13  }
14 }
15
16 };
17
18 car.start();
19
20 // Adding method stop() Later to the object
21
22 car.stop = function() {
23   console.log('Applying Brake...');
24 }
25
26 }
27
28 car.stop();
```

The right sidebar shows the 'Output' section with the following text:

```
Starting the engine...
Applying Brake...
```

Creating object with Object.create() Method

The screenshot shows the OneCompiler web interface. The editor contains the following JavaScript code:

```
1 const coder = {
2
3   isStudying : false,
4
5   printIntroduction : function(){
6
7     console.log('My name is ${this.name}. Am I studying?: ${this.isStudying}');
8
9   }
10 }
11
12 };
13
14 const me = Object.create(coder);
15 me.name = 'Mukul';
16 me.isStudying = true;
17
18 me.printIntroduction();
```

The right sidebar shows the 'Output' section with the following text:

```
My name is Mukul. Am I studying?: true
```

Using es6 classes

The screenshot shows the OneCompiler web interface. The editor contains the following JavaScript code:

```
1 // Using es6 classes
2
3 class Vehicle {
4
5   constructor(name, maker, engine) {
6
7     this.name = name;
8
9     this.maker = maker;
10
11     this.engine = engine;
12   }
13 }
14
15
16
17
18 let car1 = new Vehicle('GT', 'BMW', '1998cc');
19
20
21 console.log(car1.name); //GT
```

The right sidebar shows the 'Output' section with the following text:

```
GT
```